Privacy-Preserving Machine Unlearning in Federated LLMs: A Certified Right-to-Be-Forgotten Framework

G.SHASHANK
Roll No. 23K91A6761
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

B.SATHISH
Roll No. 23K91A6719
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

G.RAKESH
Roll No. 23K91A6759
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

B.AKHIL
Roll No. 23K91A6721
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

[GUIDE NAME]
[DESIGNATION], Project Guide
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

***Abstract*—**Federated language models keep raw data on client devices, yet every client's influence stays in the shared parameters, so a right-to-be-forgotten request can be met only by retraining or by an unverifiable heuristic. This paper presents ForgetSafe, a reproducible demonstrator of certified client-level unlearning for federated language models. Each client's clipped parameter update is tracked; a deletion request subtracts that update from the noisy aggregate, and the result is checked against a retained-only recomputation, a membership-inference attack, a utility measure and a hash-chained audit ledger, which together issue a certificate. The model is a character-bigram proxy for an LLM adapter, trained on 25 sentences held by five simulated clients. Over 250 requests, removal is exact by construction (distance 3e-14) and utility retention is 97.1 %, but the default noise leaves the model worse than a uniform one (perplexity 142) and the attack at chance (AUC 0.52). Without noise, attack AUC falls from 0.99 to 0.54 after removal. We report these limits, the audit ledger's blind spots and the route to real LLM adapters.

***Index Terms*—**audit trail, differential privacy, federated learning, machine unlearning, membership inference, right to be forgotten.

I. INTRODUCTION

Machine unlearning for federated large language models (LLMs) is a young field. Following the review method of the base paper [1], which organises a research area into a taxonomy, compares the strengths and limits of each class and states the open gap, this paper surveys the area (Section II) and then builds and measures a small certified-unlearning framework (Sections III and IV). The base paper itself reviews sign-language recognition, so only its method, not its subject, is used.

In federated learning, clients train locally and send only parameter updates to a server [2]; parameter-efficient adapters such as LoRA [3] make this practical for LLMs. Raw text never leaves a client, but the client's influence stays in the shared parameters, and large models can reproduce training text [4]. Data-protection law gives individuals a right to erasure [5], so a client may demand that its contribution be removed. Today this is done in one of three ways: retraining from scratch, which is exact but costs a full training run per request; retraining only affected shards [6]; or approximate weight edits that come with no proof [7]. Certified removal exists for convex, centralised models [8, 9], but a federated pipeline that removes a client, proves the removal and leaves an audit record that a regulator can check is still missing.

The study asks four questions. Does removal by subtraction give the same model as retraining? Does the certificate's own check mean anything? What do the differential-privacy noise and its bound do to utility and to leakage? And can the audit ledger be trusted by a regulator? The answers are measured on a small, fully reproducible system, so that every number can be recomputed in minutes on an ordinary PC.

The contributions of this work are:

1) a tracked-update workflow: clipped client updates are stored, and removal is the subtraction of one stored update, with no raw data and no retraining;
2) a verification stack that issues a certificate per request: retained-only recomputation, a loss-based membership attack, utility retention and a SHA-256-chained ledger;
3) an evaluation that tests the certificate itself (50 seeds, a noise sweep, a fresh-noise retrain comparison, a noise-free attack and an audit-tamper matrix);
4) a plain account of what is not shown: the model is a bigram proxy, the default noise swamps the signal, and the ledger has blind spots.

Section II surveys related work, Section III describes the method, Section IV reports measurements and Section V concludes.

A. System Hardware

All measurements ran on one workstation: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11 Pro. No GPU was used.

B. Software Requirements

The backend uses only the Python 3.11.9 standard library (http.server, hashlib, unittest). The interface is React 19 with Vite 8 and Recharts on Node.js 24.19; screens were captured with Playwright in Chrome.

II. LITERATURE SURVEY

A. Base Paper

Khan et al. [1] review deep-learning approaches to continuous sign language recognition (IEEE Access, 2025,

21 pages). They organise the studies into a taxonomy covering sign language, data collection, input method, gesture signals, identification methods and datasets, split the models into spatial, temporal and alignment approaches with their benefits and drawbacks, and close with challenges such as real-time processing, dataset diversity and signer variability. Its subject is unrelated to unlearning, so it offers no result that ForgetSafe could be compared with; what is taken from it is the taxonomy, compare, find-the-gap method used in this section.

B. Related Work

Exact and sharded removal. Cao and Yang [10] make a learner forget by updating the sums it is built from, and SISA training [6] confines retraining to the shard that held the data. *Certified removal.* Guo et al. [8] remove a point from a convex model with a Newton step plus noise and an (ε, δ) guarantee; Sekhari et al. [9] bound how many deletions a learner can absorb. *Federated unlearning.* FedEraser [7] keeps the clients' historical updates on the server and calibrates them to rebuild a model without one client, which is the idea ForgetSafe reduces to its simplest form. A recent survey [11] compares FedEraser, FedCIO, FedRecovery and F2UL and finds a trade-off between removal precision and cost, and calls for standard evaluation metrics. *Recent systems.* FedMemPrune [12] argues that unlearning should remove only the information unique to the forgotten data and resets the parameters that memorise it. Oblivionis [13] unifies federated learning and unlearning for LLMs and compares six federated and five unlearning algorithms. FedORA [14] casts removal in vertical federated learning as a primal-dual problem with a bound to retraining. *Privacy tools.* The Gaussian mechanism [15] and its tighter calibration [16] give the (ε, δ) bounds, and loss-based membership inference [17, 18] is the standard empirical check. Three further items of the project's literature list could not be retrieved and are not used.

Strengths and limits. Retraining and sharding give exact removal, but their cost grows with model size and request rate. Certified removal gives a formal guarantee, but only for convex models. Historical-update methods are cheap but approximate. The memorisation-aware and primal-dual methods target neural networks and report agreement with retraining on benchmarks, not a per-request proof, and the LLM framework [13] concentrates on measuring forgetting and utility rather than on evidence a regulator can check. The survey [11] names the needs that remain open: standard metrics, efficiency and non-IID data. The privacy literature adds a warning that matters here: a differential-privacy statement describes a release mechanism, and it says nothing about forgetting unless the noise and the accounting are handled again after the removal.

TABLE I
RELATED WORK COMPARED

Work	Approach	Data	Reported result	Limitation
Khan [1]	Taxonomy review of sign-language models	Sign-language datasets	None on unlearning	Unrelated subject; method only
FedMemPrune [12]	Prune parameters that memorise	CIFAR-10/100, EMNIST	Close to retraining baselines	Image models, not LLMs
Oblivionis [13]	FL + unlearning framework, 6 + 5 algorithms	Llama-2/3 models; TOFU, MUSE	Beats local training; balance of forgetting and utility	No audit or certificate stage described
FedORA [14]	Primal-dual removal, bound to retrain	Five tabular and image sets (vertical FL)	Comparable to retraining at lower cost	Vertical FL; no audit record
Zhang survey [11]	Review of FedEraser, FedCIO, FedRecovery, F2UL	Review (no new data)	Precision-cost trade-off	Review only; no system
Guo [8]	Newton step + noise	Convex models	(ε, δ) removal guarantee	Not federated, not LLM
ForgetSafe (this work)	Tracked-update subtraction + certificate + ledger	25 sentences, 5 clients	Exact removal; certificate tested (Sec. IV)	Bigram proxy; noise-dominated default

Gap. The works above remove influence heuristically, with an empirical or a convex-model guarantee; none ties a federated removal to a certificate, an independent check and a tamper-evident record, and none asks whether such a certificate is itself sound. ForgetSafe addresses the first part at demonstrator scale and the second part by testing its own certificate. The project's own survey lists the same existing practice and its limits: full retraining is too expensive for large federated LLMs, gradient-ascent or weight-adjustment heuristics give no formal guarantee, sharded training adds architectural complexity, differentially private training limits individual influence but costs utility, and unlearning is generally not verifiable or auditable for a regulator and mostly targets centralised rather than federated LLM models. Those five limits frame the four questions in Section I.

III. METHODOLOGY

A. Pipeline

Fig. 1 shows the system and Fig. 2 the main flow. A Python standard-library server (port 8109) holds one engine; a React interface (port 5109) calls nine JSON endpoints: health, state, train, session, unlearn, generate, audit verify, latest certificate and the legacy page. Training builds the noisy global model; an unlearning request removes one client and runs the checks; every training and unlearning event is appended to the ledger.

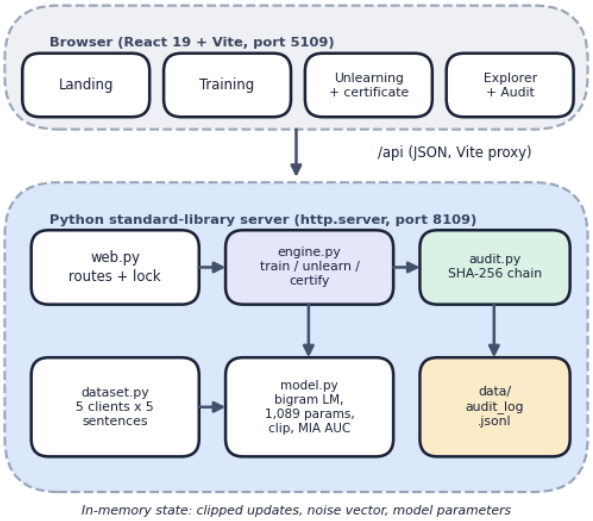


Fig. 1. ForgetSafe architecture: React interface, standard-library server, engine, model and hash-chained ledger.

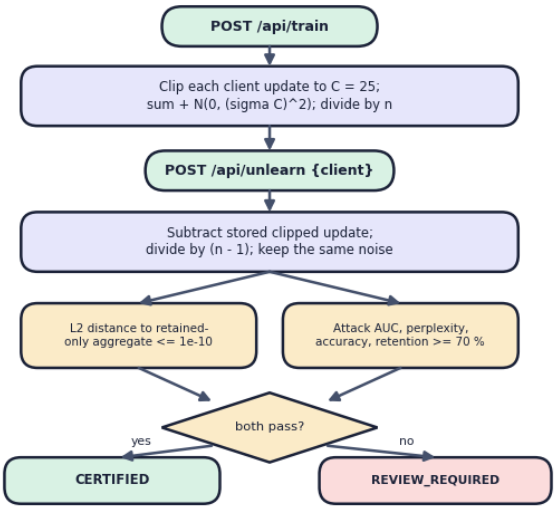


Fig. 2. Main flow: train, unlearn, two checks, certificate status.

B. Data and Model

Five simulated clients (Finance, Healthcare, Education, Retail, Travel) hold five hand-written sentences each, 25 records in all; five public sentences measure utility and five unseen sentences serve as non-members. No external dataset is used. Text is lower-cased to a 33-symbol alphabet with start and end markers. The model is a character bigram: a client's update u is the vector of its 33 × 33 = 1,089 bigram counts, standing in for an LLM adapter update, and the next-character distribution is $P(c'|c) \propto \max(0, \theta[c, c']) + 0.15$.

C. Formulation

With C = 25, noise multiplier σ = 3 and n clients, training releases

$$\hat{u}_i = u_i \cdot \min(1, C / \|u_i\|) \quad (1)$$

$$\theta = (\Sigma_i \hat{u}_i + z) / n, \quad z \sim N(0, \sigma^2 C^2 I) \quad (2)$$

$$\varepsilon = \sqrt{(2 \ln(1.25/\delta))} / \sigma, \quad \delta = 1e\text{-}5 \quad (3)$$

giving ε = 1.615. A request for client j stores nothing new and computes

$$\theta_{-j} = (\Sigma_i \hat{u}_i + z - \hat{u}_j) / (n - 1) \quad (4)$$

$$\Delta_j = \| \theta_{-j} - (z + \Sigma_{i \neq j} \hat{u}_i) / (n - 1) \|_2 \leq 1e\text{-}10 \quad (5)$$

Because the aggregate is linear, Δ is zero up to floating-point error: the certificate's reference check is an identity, not an independent retrain, since it reuses the stored noise z. The status is CERTIFIED only if Δ passes and utility retention stays at or above 70 %, where utility = 0.65·accuracy + 0.35/(1 + ln(1 + perplexity)) and retention is the ratio after to before.

D. Why This Algorithm

Subtracting a stored update was chosen because it needs no raw data, costs O(d) in the parameter count d, and is exact for a linear aggregate; the price is that the server must store every client's clipped update, itself a sensitive artefact. For multi-round nonlinear LLM fine-tuning an update influences later updates, so exact subtraction would no longer hold and methods such as [7, 12] become necessary. The Gaussian noise gives the one-shot release bound in (3); it is the classical calibration [15], which is formally valid only for ε < 1. Adding or removing one client changes Σû by at most C in L2, so noise of standard deviation σC per coordinate is the Gaussian mechanism with L2 sensitivity C, and the model θ and anything computed from it are post-processing of that one release. The unlearned model reuses z: it is a second function of the same noisy release, not a fresh one. That is why ε in (3) is reported unchanged after a removal and why it is not, by itself, a statement about forgetting.

E. Verification and Audit

A bigram proxy was chosen because it keeps what matters for the question, a linear federated aggregate, clipping, noise, an attack and an audit record, and removes what would hide it, namely GPU training time and nonlinearity, so the whole stack runs offline on a classroom PC and the full measurement suite finishes in about three minutes.

The attack scores a text by its negative loss and reports the AUC of the client's five records against five non-members [17, 18]; 0.5 means no leakage. Each ledger record stores the SHA-256 of the previous hash and its canonical JSON, so verification re-walks the chain. The software has five unit tests and a 14-check end-to-end smoke test; the measurement suite adds 11 experiments.

$$h_k = \text{SHA-256}(h_{k-1} \mid \text{canonical}(r_k)), \quad h_0 = 64 \text{ zeros} \quad (6)$$

A certificate has 25 fields: identifiers, status, method, ε, δ, C, σ, the residual and reference distances, the attack AUC, perplexity and accuracy before and after, retention, client counts, model hashes before and after, and the hash of its own ledger record h. It is returned by the API and can be downloaded as JSON.

F. Algorithms and Threat Model

Algorithm 1 (train). 1) Count the bigrams of each client's normalised records to obtain u; 2) clip to norm C and store the clipped update with its raw norm and scale; 3) draw z

once from $N(0, \sigma^2 C^2 I)$ with the session seed and store it; 4) keep the sum $S = \Sigma\hat{u} + z$; 5) release $\theta = S/n$, evaluate perplexity and accuracy on the public sentences, hash θ with SHA-256 and append a FEDERATED_TRAINING record.

Algorithm 2 (unlearn client j). 1) Reject the request if the model is untrained, the client is unknown or already removed, or only one client remains; 2) measure the metrics and the attack AUC on j's records; 3) set $S \leftarrow S - \hat{u}_j$, drop j from the active set and $\theta \leftarrow S/(n-1)$; 4) measure again; 5) compute Δ against the retained-only aggregate; 6) set the status and append a CLIENT_UNLEARNING record that carries every certificate field; 7) return the certificate with its record hash.

Threat model. The server is trusted to hold the clipped updates. The questioner is a party who later asks whether a removed client's text is still in the model (membership inference) or an auditor who wants a record. Malicious clients, a malicious server, secure aggregation and any adversary with write access to the ledger file are out of scope.

G. Implementation

The server wraps one engine in a re-entrant lock, so requests are serialised and every caller shares one session; POST /api/session replaces the engine but keeps the ledger. The React interface has six pages (landing, dashboard, training, unlearning, explorer and audit) that call the same endpoints through a Vite proxy, show the certificate with a before-and-after chart and the ledger table, and download the certificate as JSON. The original single-page interface and a Tkinter client remain available.

H. Evaluation Protocol

Every experiment runs the real engine on a throw-away ledger. The seed study repeats the default configuration for 50 seeds and removes each of the five clients in turn (250 requests). The sweep varies σ over 12 values from 0.001 to 10 with 20 seeds each. The retrain comparison draws, for each of 40 seeds and five clients, a fresh-noise retrain on the retained clients and a second independent one, at $\sigma = 3$ and $\sigma = 0.5$. The noise-free run sets $\sigma =$ 1e-6. The scaling run uses synthetic clients drawn from a 20-word vocabulary, from 5 to 500 clients, with the median of five repeats; the baseline is the proxy training on the retained clients. API timings are taken on 127.0.0.1 over 30 to 100 calls per endpoint. The tamper run applies seven edits to a three-record ledger. All randomness is seeded.

IV. RESULTS

Conditions. Default configuration is seed 2026, $C = 25$, $\sigma = 3$, $\delta =$ 1e-5; seed studies use 50 seeds × 5 clients (250 requests); the noise sweep uses 20 seeds per σ; the retrain comparison uses 200 pairs per σ; the noise-free run uses $\sigma =$ 1e-6. Perplexity is on five public sentences. The seed study measures spread, the sweep measures the privacy-utility trade-off, the noise-free run shows what the removal changes when nothing hides it, the fresh-noise comparison tests the certificate against a real retrain, and the tamper and scaling runs probe the ledger and the cost.

TABLE II
MEASURED UNLEARNING RESULTS (MEAN OVER REQUESTS)

Setting	ε	Perplexity	Accuracy	AUC before → after	Retention %
Default, seed 2026 (n = 5)	1.61	130.2	5.1 %	0.22 → 0.21	96.7
Default, 50 seeds (n = 250)	1.61	142.2±28	3.7 %	0.52 → 0.50	97.1
$\sigma = 0.1$ (20 seeds)	48.4	16.3	23.5 %	0.86 → 0.55	96.5
$\sigma = 1$ (20 seeds)	4.84	65.6	5.4 %	0.60 → 0.52	91.6
$\sigma = 4.85$ (20 seeds)	1.00	197.9	3.4 %	0.54 → 0.53	97.7
Noise-free $\sigma =$ 1e-6 (n = 5)	≈ 5e6	12.4	n/a	0.99 → 0.54	99.7

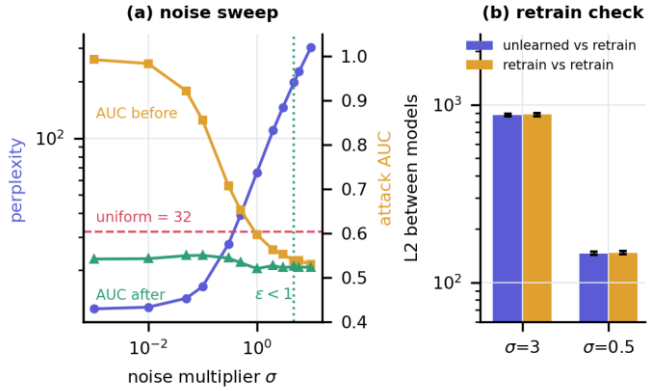


Fig. 3. (a) Perplexity and attack AUC against the noise multiplier (20 seeds). (b) L2 distance of the unlearned model to a fresh-noise retrain and between two retrains (200 pairs).

Removal and utility. Removal is exact to 3.2e-14 and all 250 requests are certified. Retention averages 97.1 % (minimum 79.9 %); removing four of five clients raises perplexity from 130 to 303 because the noise stays while the signal is divided by fewer clients; every step is certified and the ledger verifies, and the fifth removal is refused with the message that at least one client must remain. At the default seed the attack AUC before removal is 0.20, 0.48, 0.08, 0.32, 0.04 for the five clients (after: 0.16, 0.44, 0.12, 0.28, 0.04); with five members and five non-members one pair moves the AUC by 0.04, and values far below 0.5 are as uninformative as values above it.

The default model is noise-dominated. The noise has standard deviation 75 per coordinate against a clipped client update of norm 25 spread over 1,089 coordinates. Perplexity is 142, worse than the uniform model's 32, and next-character accuracy is 3.7 %. The attack is already at chance before removal (AUC 0.52 ± 0.22), so the mean change of -0.03 is within noise. Useful models (perplexity < 20) need $\sigma \le 0.1$, i.e. $\varepsilon \ge 48$, while the classical bound is valid only for $\varepsilon < 1$ ($\sigma > 4.84$), where perplexity is 198 or more: no setting is both private by this bound and useful. Between $\sigma = 0.001$ and 0.1 accuracy stays at 23.5-24.6 % and perplexity at 12.5-16.3; at $\sigma = 0.3$ mean retention dips to 88.1 % and one of the 100 requests falls below the 70 % floor and is marked REVIEW_REQUIRED, the only such case in the sweep. Because the retention floor is measured against a model that is already near-useless at $\sigma = 3$, it cannot fail there: none of the 250 default requests did.

Is the removal real? Without noise the attack separates members almost perfectly (AUC 0.99) and removal brings it to 0.54 with 99.7 % retention, so the subtraction does erase

the signal. Against a true retrain with fresh noise, the unlearned model lies 875 ± 16 away at σ = 3 (146 at σ = 0.5), the same as two independent retrains are from each other (881 and 147). That distance is pure noise, which supports the claim that the unlearned model is distributed like a retrained one, but the 1e-14 certificate value alone does not.

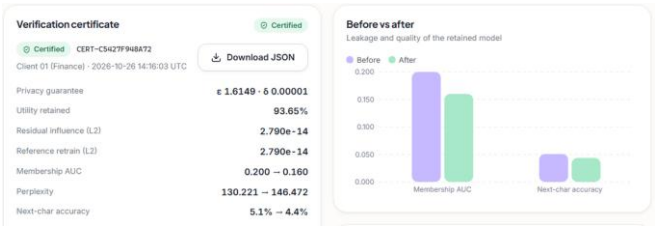


Fig. 4. Output screen: certificate for Client 01 (Finance), ε 1.6149, AUC 0.200 → 0.160, retention 93.65 %.

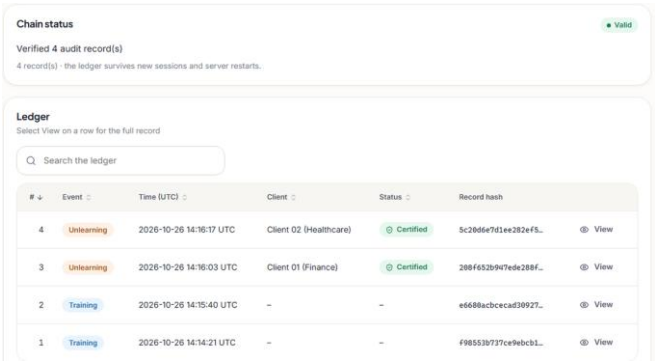


Fig. 5. Output screen: compliance ledger, four chained records and a valid hash chain.

TABLE III
MEDIAN COST OF UNLEARNING AND OF RETRAINING THE PROXY

Clients	Records	Train ms	Unlearn ms	Retrain ms	Retrain / unlearn
5	25	14.0	45.3	13.4	0.30×
25	500	32.3	72.0	30.5	0.42×
100	5,000	159.6	115.8	159.5	1.38×
250	25,000	691.2	195.8	709.6	3.62×
500	50,000	1425.6	204.0	1384.8	6.79×



Fig. 6. Audit-ledger tamper matrix: seven attacks on a three-record ledger; green = detected, red = not detected.

Audit and service behaviour. The ledger detects an edited field, a flipped status, a deleted record and swapped records, but misses tail truncation, a full re-hash forgery and an appended forged record (no signature or anchor). An append re-reads the file: 0.44 ms at one record and 19.6 ms at 2,000. Unlearning (45 ms) is slower than re-running the proxy training (13 ms) with 5 clients, 1.4 times faster at 100 clients and 6.8 times faster at 500; this says nothing about LLM-scale retraining. On localhost the median latency is 36 ms for training and 58 ms for unlearning. A 2 MB body aborts the connection instead of returning an error, unsupported methods return an HTML page, and no endpoint is authenticated.

Testing. The five unit tests and the 14 smoke checks pass. 20 further edge-case calls (malformed JSON, a JSON array body, an unknown and an SQL-like client id, an empty, Unicode and 5,000-character seed, a repeated removal, removal of the last client, a path-traversal URL) return JSON errors with status 400 or 404 or succeed as intended, apart from the oversized body and the unsupported method noted above. Not tested: concurrent load, a real LLM, GPU training and any penetration test.

Observed limits. The attack has a resolution of 0.04 and uses only ten texts; state lives in memory and is lost on restart while the ledger persists; one engine is shared by every caller; the 25 sentences cannot show generalisation; and exactness of subtraction holds for one aggregation round only. These limits, not the arithmetic, bound what the certificate can claim.

Comparison. The base paper [1] reports no comparable metric. FedMemPrune [12], Oblivionis [13] and FedORA [14] use image, benchmark-LLM and vertical-federated data with neural models, so none of the numbers above is comparable with theirs; the comparison is qualitative: ForgetSafe is exact only because its aggregate is linear and single-round, whereas those works handle nonlinear networks. It adds a per-request certificate and a tamper-evident ledger; FedORA certifies through a bound to retraining, and the other two works describe no audit record.

Implications. Four practical rules follow. Report utility next to ε, because a bound that leaves the model worse than uniform protects nothing useful. Test a certificate against a fresh-noise retrain and a noise-free attack, because an identity check cannot fail. Draw fresh noise or re-account the budget after a removal, because reused noise makes the reported ε a statement about training, not about forgetting. Treat a hash chain as evidence of casual edits only; signatures or an external anchor are needed to detect truncation or re-hashing.

V. CONCLUSION

ForgetSafe meets its objectives on a bigram proxy: federated aggregation across five clients (partly, no LLM), parameter-level removal without retraining (met), a differential-privacy bound (partly: ε = 1.61 lies outside the range of the classical bound, the noise is reused and no accountant exists), attack and retrain verification (met, but the attack is uninformative at the default noise), utility retention (97.1 %, on a near-useless baseline) and per-request audit records (met, with the blind spots above). The key finding is that the certificate proves exactness of the arithmetic, not forgetting of an LLM; the fresh-noise and noise-free experiments supply the evidence the certificate cannot. Future work: replace the proxy with LoRA adapters of a small Hugging Face model over several federated rounds [3]; add privacy accounting [16] and fresh noise after removal; sign and externally anchor the ledger; and test

stronger membership attacks. Until then ForgetSafe is a reproducible demonstrator of the workflow (train, remove, check, certify, record) and of how to test such a certificate, not a compliance product and not a claim that an LLM has been formally erased. Every number in this paper is regenerated by one script from fixed seeds.

In answer to the four questions: removal by subtraction equals the retained-only aggregate exactly and matches a fresh retrain in distribution; the certificate's own check is an identity and cannot fail; the default noise removes the leakage and the utility together; and the ledger resists casual edits but not an adversary who can rewrite the file.

REFERENCES

[1] A. Khan, S. Jin, G.-H. Lee, G. E. Arzu, L. M. Dang, T. N. Nguyen, W. Choi, and H. Moon, "Deep learning approaches for continuous sign language recognition: A comprehensive review," IEEE Access, vol. 13, pp. 55524-55544, 2025, doi: 10.1109/ACCESS.2025.3554046.

[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS), PMLR vol. 54, 2017, pp. 1273-1282.

[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in Proc. Int. Conf. Learning Representations (ICLR), 2022.

[4] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in Proc. 30th USENIX Security Symp., 2021, pp. 2633-2650.

[5] European Parliament and Council, "Regulation (EU) 2016/679 (General Data Protection Regulation)," Official Journal of the European Union, L 119, May 2016, Art. 17 (right to erasure).

[6] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in Proc. IEEE Symp. Security and Privacy (SP), 2021, pp. 141-159.

[7] G. Liu, X. Ma, Y. Yang, C. Wang, and J. Liu, "FedEraser: Enabling efficient client-level data removal from federated learning models," in Proc. IEEE/ACM 29th Int. Symp. Quality of Service (IWQoS), 2021, pp. 1-10.

[8] C. Guo, T. Goldstein, A. Hannun, and L. van der Maaten, "Certified data removal from machine learning models," in Proc. 37th Int. Conf. Machine Learning (ICML), PMLR vol. 119, 2020, pp. 3832-3842.

[9] A. Sekhari, J. Acharya, G. Kamath, and A. T. Suresh, "Remember what you want to forget: Algorithms for machine unlearning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021, pp. 18075-18086.

[10] Y. Cao and J. Yang, "Towards making systems forget with machine unlearning," in Proc. IEEE Symp. Security and Privacy (SP), 2015, pp. 463-480.

[11] W. Zhang, "A comprehensive investigation of federated unlearning: Challenges, methods and future prospects in privacy-sensitive applications," in Proc. 2nd Int. Conf. Data Analysis and Machine Learning (DAML), SciTePress, 2024, pp. 569-573, doi: 10.5220/0013528500004619.

[12] J. Wei, Y. Zhang, H. Zhang, L. Y. Zhang, C. Chen, K.-L. Ong, J. Zhang, and Y. Xiang, "Rethinking federated unlearning via the lens of memorization," in Proc. 32nd ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD '26), Jeju Island, South Korea, Aug. 2026, doi: 10.1145/3770855.3817785; arXiv:2605.24545.

[13] F. Zhang, X. Yan, T. Wu, W. Li, T. Chen, Y. Cao, R. Yan, L. Huang, W. Y. B. Lim, and Q. Yang, "Oblivionis: A lightweight learning and unlearning framework for federated large language models," arXiv:2508.08875, Aug. 2025.

[14] Y. Jiang, X. Tong, Z. Liu, X. Zhang, K.-Y. Lam, and C. W. Tan, "Certifying the right to be forgotten: Primal-dual optimization for sample and label unlearning in vertical federated learning," arXiv:2512.23171, 2025.

[15] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," Foundations and Trends in Theoretical Computer Science, vol. 9, no. 3-4, pp. 211-407, 2014.

[16] B. Balle and Y.-X. Wang, "Improving the Gaussian mechanism for differential privacy: Analytical calibration and optimal denoising," in Proc. 35th Int. Conf. Machine Learning (ICML), PMLR vol. 80, 2018, pp. 394-403.

[17] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in Proc. IEEE Symp. Security and Privacy (SP), 2017, pp. 3-18.

[18] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in Proc. IEEE 31st Computer Security Foundations Symp. (CSF), 2018, pp. 268-282.